Design and Empirical Evaluation of an Event-Driven Outbox and CQRS Architecture for High-Concurrency Courier Logistics

Võ Văn Tú Tài, Trần Tấn Tài, and Huỳnh Nam
Faculty of Information System, Industrial University of Ho Chi Minh City
22655911.tai@student.iuh.edu.vn, 22727771.tai@student.iuh.edu.vn

***Abstract*—** Express courier platforms endure intense burst write amplification across barcode scans, waybill transitions, and real-time analytics. Shared relational schemas suffer row-level lock contention under concurrent load, while naive dual-writes risk losing domain events on process crashes. We design and empirically evaluate the Nexus Event-Driven Outbox and Command Query Responsibility Segregation (CQRS) architecture built on NestJS 10 microservices, isolated PostgreSQL 16 databases, and RabbitMQ 3.13. Our implementation integrates: (1) a dual-layer idempotency guard pairing fast-path key lookup with PostgreSQL `P2002` unique-constraint recovery; (2) persistent outbox relays contrasting HTTP Management API publishing against native AMQP `ConfirmChannel` transport; (3) a deterministic 19-state shipment finite-state machine (FSM) whitelist; (4) decoupled tracking and 8-way KPI read projections; and (5) multi-tier Dead-Letter TTL retry queues (`10 s` → `1 min` → `.dlq`) governed under a structured 5-level human-control scale classifying operational governance rather than psychometric user ratings. Live benchmarks on the running cluster show 7.52× write throughput scaling to 783.36 req/s (c = 25), a 65.59% reduction in p_{95} tracking read tail latency under c = 50 concurrent writers (15.73 vs. 45.73 ms, $p = 4.83 \times 10^{-67}$), a 28.25× publish speedup in a pipelined batch-50 AMQP probe (2.84× sequential relay), 150/150 exact-once writes across 500 duplicate HTTP calls, 60/60 (100.0%; 95% CI [94.04%, 100.0%]) FSM invariant preservation against out-of-order attacks, and 5/5 ([47.82%, 100.0%]) self-healing at 10.36 s alongside 5/5 poison quarantine to `.dlq`, while exposing three active code findings (`F1`–`F3`) with targeted engineering remediations.

***Index Terms*—** Event-Driven Architecture, Transactional Outbox Pattern, Command Query Responsibility Segregation (CQRS), Idempotency, RabbitMQ Dead-Letter Exchange, 5-Level Human-Control Scale, Courier Logistics.

I. INTRODUCTION

Modern express courier networks generate continuous streams of domain events across sorting hubs, handheld Personal Digital Assistants (PDAs), dispatch consoles, and public tracking portals. During promotional surges, concurrent inbound, outbound, and delivery scans create sharp write bursts alongside latency-sensitive read queries from merchants and recipients [1], [5]. Consolidating command ingestion, multi-table state validation, and multi-dimensional KPI rollups within one shared ACID transaction induces row-level lock contention, pool saturation, and tail-latency spikes.

Decomposing the platform into autonomous bounded-context microservices avoids single-schema coupling [6], [7], yet introduces four practical data management hazards examined empirically in our graduation thesis implementation:

1) **Dual-Write Consistency Anomaly:** Updating a service's local database and publishing to a broker outside a coordinated persistence flow risks permanently lost events or phantom downstream states on process crashes [1].
2) **Outbox Dwell & Transport Bottlenecks:** While the Transactional Outbox guarantees at-least-once emission, polling intervals (T_{poll}), batch caps (B), and HTTP Management API serialization dominate convergence lag.
3) **Retry Storms & Consumer Race Conditions:** Unstable wireless PDA links emit sequential and parallel duplicate writes, whereas asynchronous broker delivery can deliver child scan events before parent waybills materialize.
4) **Out-of-Order State Skew & Operator Control:** Delayed or reordered events reaching terminal shipments (`DELIVERED`, `CANCELLED`) can regress states unless guarded deterministically alongside clear human escalation for dead-letter faults [16].

Without heavyweight two-phase commit (2PC) or Kafka cluster overhead, we evaluate the live Nexus Event-Driven Outbox and CQRS platform across three research questions: (RQ1) How much does Outbox + CQRS decoupling improve write scaling and isolate public tracking read tail latency (p_{95}) compared with a synchronized RDBMS baseline? (RQ2) Which pipeline stage dominates cross-database convergence lag, and how much faster is native AMQP `ConfirmChannel` batching than sequential HTTP publishing? (RQ3) How reliably do dual-layer idempotency, the 19-state FSM whitelist, and 5-level human-control TTL/DLQ governance preserve integrity—and what active code defects surface under faults?

II. RELATED WORK

Distributed Data Management & Microservices: The CAP theorem [3] and PACELC formulation [2] frame trade-offs between strong consistency and low-latency availability, motivating BASE workflows [4]. Empirical studies by Laigner et al. [5] show that database-per-service decomposition [6], [7] shifts consistency burdens from SQL engines into messaging handlers, requiring strict verification of idempotency, single-writer ownership, and asynchronous fault handling.

Transactional Outbox & Message Brokers: Hohpe and Woolf [8] and Richardson [1] establish the Transactional Outbox and Polling Publisher patterns to prevent dual-write event loss without distributed XA locks [9], [15]. While partitioned logs such as Apache Kafka [10], [11] target high-volume replay, AMQP brokers like RabbitMQ [12] fit polyglot service backbones via flexible topic routing and native Dead-Letter Exchange (DLX) TTL queues.

CQRS, Idempotency & Ordering Safety: CQRS [13], [17] isolates write commands from denormalized read views optimized for point lookups and analytics. Helland [14] emphasizes idempotent receivers keyed by unique operation tokens, while Lamport [16] highlights logical ordering hazards across asynchronous messages. Rather than relying on abstract queue simulators, this work audits our live NestJS + PostgreSQL 16 + RabbitMQ 3.13 cluster under disclosed synthetic bursts and fault workloads, surfacing active codebase vulnerabilities (`F1`–`F3`) and targeted engineering fixes.

III. SYSTEM ARCHITECTURE AND CORE MECHANISMS

A. Bounded-Context Topology and Service Separation

As shown in Fig. 1 and Table I, the Nexus backend deploys 12 isolated PostgreSQL 16 domain databases across NestJS 10 TypeScript microservices, centering this study on four core event-driven contexts alongside companion dispatch, manifest, delivery, and payment stores: `ScanService` (`scan_db`), `ShipmentService` (`shipment_db`), `TrackingService` (`tracking_db`), and `ReportingService` (`reporting_db`). Single-writer ownership is strictly enforced (`scan_db` owns `CurrentLocation`; `shipment_db` holds authoritative `Shipment` state; `tracking_db` serves public reads; `reporting_db` maintains 8-way KPI rollups), while Redis 7 in the gateway BFF is scoped strictly to WebSocket chat pub/sub with zero role in outbox or CQRS. Services exchange persistent events (`delivery_mode = 2`) over RabbitMQ's durable `domain.events` topic exchange (65 active queue bindings).

TABLE I
NEXUS BOUNDED-CONTEXT SERVICES, SCHEMA OWNERSHIP, AND MESSAGING CONTRACTS

Service / DB	Owned State / Model	Relay / Consumer Mode	Safety & Dedup Contract
`ScanService` (`scan_db`)	**`ScanEvent`, `OutboxEvent`, `CurrentLocation`**	**HTTP Mgmt Relay (T_{poll} = 1,000 ms, B = 50)**	**`IdempotencyRecord` + `ScanEvent` `P2002`**
`ShipmentService` (`shipment_db`)	**`Shipment` (19-state FSM), `outbox_events`**	**AMQP Confirm Outbox + Push (`noAck: false`)**	**`VALID_TRANSITIONS`; `.retry.10s`/`.1m` → `.dlq`**
`TrackingService` (`tracking_db`)	`TimelineEvent`, `TrackingIndex`, `TrackingCurrent`	AMQP Push Consumer (`tracking-service.q`)	`TimelineEvent.eventId` `P2002` deduplication
`ReportingService` (`reporting_db`)	`KpiDaily`, `KpiMonthly` (8-way), `AggregationJob`	HTTP Pull (1,000 ms, B = 20, `ack_requeue_false`)	`AggregationJob.jobKey` `P2002` deduplication
`CoupledSyncBaseline` (shared PG DB)	Monolithic SQL tables + 8-way KPI row counters	Single-TX SQL ACID (PG pool = 21)	Synchronous row locks + multi-table SQL JOINs

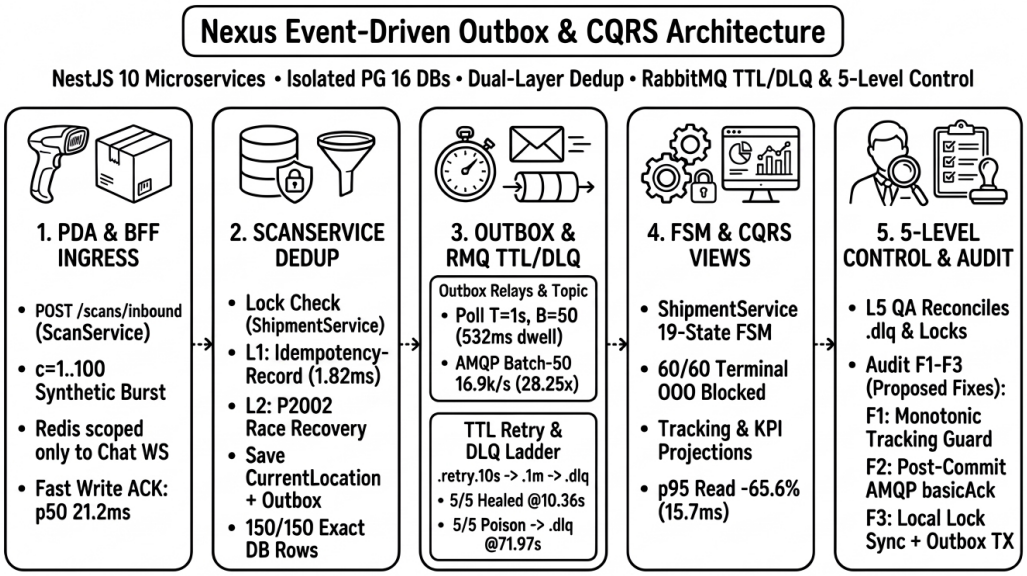


Fig. 1. Architecture of the live Nexus Event-Driven Outbox and CQRS pipeline across isolated PostgreSQL 16 databases and RabbitMQ 3.13, illustrating ingress and `ScanService` *dual-layer deduplication, outbox relays and TTL/DLQ recovery,* `ShipmentService` *19-state FSM governance, decoupled CQRS projections, and Level-5 operator control alongside proposed code-audit remediations (*`F1`–`F3`*).*

B. Dual-Layer Idempotency and Persistent Outbox Writes

Before persisting an inbound scan, `ScanService` runs a synchronous HTTP lock check against `ShipmentService` (`assertShipmentNotLocked`), rejecting locked parcels (HTTP 400) unless an authorized audit override (`INBOUND` with note `INVENTORY_CHECK` or Vietnamese 'Kiểm tồn kho') or active reverse-logistics transit (`RETURN_STARTED`) applies. Once permitted, the handler scopes the key as `flow:idempotencyKey` and executes two-tier deduplication:

1) **Layer 1 — Fast Sequential Lookup:** Queries `IdempotencyRecord` by `scopedIdempotencyKey` and returns the cached JSON snapshot immediately (1.825 ms median) for sequential PDA retries.
2) **Layer 2 — Parallel Race Recovery (`P2002`):** Inserts into `ScanEvent` guarded by a unique constraint on `idempotencyKey`. When concurrent duplicate requests bypass Layer 1 simultaneously, PostgreSQL rejects losing inserts with Prisma unique error `P2002`, prompting `createIfAbsent()` to fetch and replay the winner's committed record.
3) **Outbox Persistence:** For each new scan, the service upserts `CurrentLocation`, appends a `PENDING` `OutboxEvent`, saves the idempotent snapshot, and records `OpsAuditLog` before returning HTTP 201 (wrapping `ScanEvent` + `OutboxEvent` inside `prisma.$transaction` is noted under finding `F3` in Section V).

C. Outbox Relays and Propagation Lag Formulation

Pending outbox records drain periodically every T_{poll} = 1,000 ms in batches up to B = 50 rows. Notably, our codebase contrasts two relay designs: `ScanOutboxRelayService` publishes sequentially via RabbitMQ's HTTP Management API, whereas `ShipmentOutboxRelayService` keeps a persistent TCP AMQP `ConfirmChannel` flushing batch entries sequentially [12]. Client-to-projection visibility lag decomposes linearly across four stages:

$$L_{E2E} = L_{ingest} + L_{dwell} + L_{broker} + L_{cons} \quad (1)$$

where L_{ingest} is HTTP command validation and ingest commit latency, L_{dwell} is outbox queue wait prior to broker confirmation, L_{broker} is broker-to-consumer transport delay, and L_{cons} denotes downstream consumer commit time (with post-commit convergence defined as $L_{conv} = L_{dwell} + L_{broker} + L_{cons}$). Because scans arrive asynchronously between periodic ticks, expected outbox dwell follows

$$L_{\text{dwell}} \approx \frac{T_{\text{poll}}}{2} + L_{\text{batch}} \quad (2)$$

where T_{poll} = 1,000 ms yields a 500 ms midpoint delay and L_{batch} denotes intra-batch serialization overhead (≈ 32 ms), closely matching the measured 531.96 ms mean dwell. When concurrent bursts exceed capacity ($N_0 > B$), outbox draining quantizes across multiple polling intervals:

$$T_{\text{drain}}(N_0) \approx \left\lceil \frac{N_0}{B} \right\rceil T_{\text{poll}} \quad (3)$$

where N_0 is the concurrent burst volume and B = 50 is the batch cap (predicting 4 ticks ≈ 3.8–4.0 s for N_0 = 200 writes).

D. 19-State Authority FSM, CQRS Views, and 5-Level Control

Downstream, `ShipmentService` consumes events via AMQP push (`noAck: false`) and guards status changes inside `ShipmentStateMachine` using an explicit `VALID_TRANSITIONS` whitelist spanning 19 lifecycle statuses and 15 consumed domain event types from `CREATED`. Three absorbing terminal states (`DELIVERED`, `RETURN_COMPLETED`, `CANCELLED`) permit only self-idempotent replays, rejecting backward transitions. Independently, `TrackingService` deduplicates `TimelineEvent` (`P2002` on `eventId`) and updates `TrackingCurrent`/`TrackingIndex`, while `ReportingService` polls via HTTP (1,000 ms, B = 20, `ack_requeue_false`) into 8-way KPI rollups (`KpiDaily`/`KpiMonthly` across `{courier, hub, zone}` × `ALL`) keyed by `AggregationJob.jobKey`. Transient errors invoke `moveToRetryOrDlq()`, ACKing the main queue immediately so healthy traffic never stalls while routing retries via `x-retry-count` across `.retry.10s` (10 s TTL), `.retry.1m` (60 s TTL), and `.dlq` (count ≥ 2) under a 5-level human-control scale categorizing system authority rather than psychometric user ratings.

IV. EXPERIMENTAL SETUP, SYNTHETIC WORKLOAD & RESULTS

A. Cluster Testbed, Synthetic Workload Criteria, and Baseline

Experiments executed live against the running microservice cluster on an Apple M4 host (10-core CPU, 16 GB RAM, macOS 15.8), Node.js v26.0.0, PostgreSQL `16-alpine` (`max_connections = 100`), and RabbitMQ `3.13-management-alpine`, restoring pre-test SQL snapshots post-run. An auxiliary synchronous baseline (`CoupledSyncBaseline`) shares the same PostgreSQL 16 engine (pool = 21, matching Prisma defaults) to execute lock checking, scan insert, timeline append, `TrackingCurrent` upsert, and 8-way `KpiDaily` updates inside one atomic SQL `BEGIN/COMMIT` transaction.

Table II discloses every synthetic cohort generated by the benchmark suite: 220 seeded synthetic shipments (`333991000001..220`) distributed round-robin across 3 hubs (`HUB_HCM_01`, `HUB_DN_01`, `HUB_HN_01`), 3 zones (`SOUTH`, `CENTRAL`, `NORTH`), and 5 couriers (`C01..C05`), standardized at `1.5 kg`, `500,000 VND`, `GENERAL`, `STANDARD` (Exp 2A staggers 120 clean scans with 60–210 ms random gaps to avoid timer aliasing). Statistical uncertainty uses two-sided Mann–Whitney U tests, 10,000-resample bootstrap 95% CIs, and exact two-sided Clopper–Pearson 95% CIs.

TABLE II
SYNTHETIC BENCHMARK WORKLOAD AND FAULT-INJECTION SPECIFICATION

Exp ID	Synthetic Pool / Volume	Construction & Stagger Rules	Measured Objective
Exp 1 / 1C	100 parcels; 150/tier; N = 200 reads (25R / 50W)	Round-robin 3 hubs × 5 couriers; 50 background writers (1C)	Write scaling crossover & CQRS read isolation (p_{95})
Exp 2A / 2B & Exp 2C	120 clean (2A); 200 burst (2B); 200 direct (2C)	Random 60–210 ms stagger (2A); c = 50 burst; direct RMQ (2C)	UUID E2E lag breakdown & HTTP vs. AMQP Confirm
Exp 3 / 3C	**500 HTTP calls (150 keys); +50 RMQ dup frames**	**100 fresh + 100 replay + 300 6× race (c = 60) + RMQ inject**	**L1/L2 `P2002` dedup & consumer `ackmode` audit**
Exp 4A / 4B	**20 locked/terminal (4A); 10 orphan faults (4B)**	**60 OOO + 20 lock probes; 5 delay @3.09 s + 5 poison (333993999*)**	**FSM terminal safety, lock TOCTOU & `.retry/.dlq`**

B. Concurrency Scaling and CQRS Read Isolation (RQ1)

Table III and Fig. 2(a)–(b) report write scaling across $c \in \{1, 10, 25, 50, 100\}$ (100% HTTP `2xx` over 750 writes/system) and concurrent tracking read isolation (N = 200 queries at 25 concurrent readers under Idle vs. c = 50 background writers).

TABLE III
CONCURRENCY SCALING (EXP 1) AND CQRS READ ISOLATION (EXP 1C) VS. SYNCHRONOUS RDBMS BASELINE

Tier / Condition	Nexus rps	Nexus p_{50}	Nexus p_{95}	Sync rps	Sync p_{50}	Sync p_{95}	Gain & Mann–Whitney p
Write c = 1 (n = 150)	104.2	6.91	19.96	423.1	1.89	3.36	Sync 4.1× (8.77×10^{-47})
Write c = 10 (n = 150)	727.3	12.66	21.41	670.0	8.38	43.54	+8.6% / -50.8% p_{95} (.00071)
Write c = 25 (Peak)	**783.4**	**29.52**	**45.89**	**597.6**	**26.99**	**108.59**	**+31.1% / -57.7% p_{95} (.177 n.s.)**
Write c = 50 (n = 150)	757.1	62.07	90.85	598.7	63.87	136.22	+26.5% / -33.3% p_{95} (.0118)
Write c = 100 (n = 150)	614.8	156.8	183.0	582.4	122.8	190.3	+5.6% / -3.8% p_{95} (.0105)
Read Idle (25R)*	—	8.46	33.70*	—	2.82	8.50	Sync faster (3.31×10^{-55})
Read (25R / 50W)	**—**	**13.03**	**15.73**	**—**	**41.26**	**45.73**	**-65.59% p_{95} (4.83×10^{-67})**

Notes: Latencies in ms; two-sided Mann–Whitney U tests over single-request latencies. *Idle Nexus p_{95} reflects cold TCP/Prisma pool warmup on early reads (mean 10.74 ± 7.33 ms idle vs. 12.60 ± 2.14 ms under write load).

Two regimes emerge in Fig. 2(a): (1) *Single-Thread Overhead* (c = 1): Coupled Sync runs faster (423.05 vs. 104.17 req/s; $p = 8.77 \times 10^{-47}$) because `ScanService` calls HTTP `assertShipmentNotLocked` and executes seven Prisma steps. (2) *Lock-Contention Inversion* ($c \geq 10$): Nexus scales 7.52× to 783.36 req/s (c = 25) while cutting write p_{95} tail latency by 57.7% (45.89 vs. 108.59 ms; p_{99} 60.84 vs. 175.53 ms)—note that the per-request median test at c = 25 is non-significant (p = 0.177) whereas c = 50 reaches p = 0.0118, while Coupled Sync plateaus near ~598 req/s (p_{99} = 233.19 ms, max = 243.06 ms at c = 100) due to shared `ALL|ALL|ALL` KPI counter row locks. Under c = 50 writers (Exp 1C, Fig. 2b), Sync joins degrade +38.45 ms in median latency (95% CI [37.89, 39.13] ms, $p = 4.83 \times 10^{-67}$), whereas CQRS views shift only +4.57 ms (CI [4.18, 4.75] ms, $p = 1.75 \times 10^{-33}$), achieving 65.59% lower p_{95} tail latency (15.73 vs. 45.73 ms) and 69.48% lower mean latency (U = 0.0, $p = 4.83 \times 10^{-67}$).

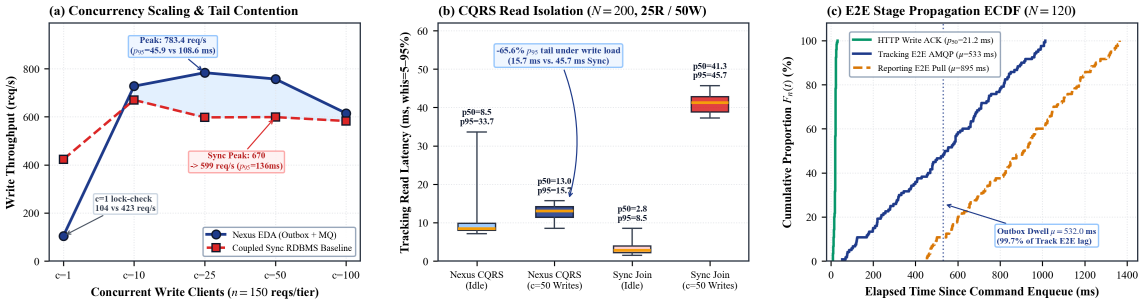


Fig. 2. Measured telemetry across live Dockerized services: (a) Write throughput scaling (c ∈{1..100}, n = 150/tier); (b) Tracking read latency distributions (5th–95th whiskers, N = 200 queries at 25 concurrent readers under Idle vs. c = 50 writers); and (c) Post-commit stage propagation ECDF (N = 120 joined scans).

C. Stage Propagation Lag & Transport Speedup (RQ2)

Correlating $N = 120$ clean synthetic events across databases via UUID `eventId` (Exp 2A, Table IV, Fig. 2c) shows Fast-Path HTTP Ingest finishes in 20.27 ms (95% bootstrap CI [19.52, 21.01] ms; $p_{50} = 21.17$ ms), while Outbox Polling Dwell averages 531.96 ms (CI [480.93, 581.24] ms; $p_{95} = 970.45$ ms)—closely matching Eq. (2) and representing 99.69% of mean post-commit Tracking convergence (99.98% of variance; ≈ 553.3 ms total client-to-view Eq. (1)). AMQP push delivery takes only 1.03 ms ([0.89, 1.18] ms) to `TrackingService` and 0.19 ms ([0.02, 0.36] ms) to `ShipmentStateMachine` (±1–3 ms container clock precision yields minor min clamping), whereas `ReportingService`'s HTTP `/get` pull adds 363.18 ms ([360.81, 365.63] ms; 895.14 ms [845.60, 942.71] post-commit E2E).

TABLE IV
STAGE LAG DECOMPOSITION (EXP 2A, $N = 120$) AND TRANSPORT MICRO-BENCHMARK (EXP 2C, $N = 200$)

Stage / Transport Channel	N	Mean [95% CI]	SD	p_{50}	p_{95}	Rate / Lag Share
Ingest HTTP ACK (L_{ingest})	120	20.27 [19.5, 21.0]	4.16	21.17	25.50	Client Write Edge
Outbox Dwell (T_{poll} = 1 s)	**120**	**531.96 [480.9, 581.2]**	**278.3**	**546.0**	**970.5**	**99.69% (99.98% Var.)**
AMQP Push → Shipment FSM	120	0.19 [0.02, 0.36]	0.92	0.00	1.00	532.15 ms Post-Commit
AMQP Push → Tracking DB	120	1.03 [0.89, 1.18]	0.77	1.00	2.00	532.99 ms Post-Commit
HTTP Pull → Reporting Cube	120	363.18 [360.8, 365.6]	13.59	360.0	387.0	895.14 ms Post-Commit
Exp 2C: HTTP Seq. (Relay A)	200	1.665 (total 333.4)	0.846	1.454	2.824	599.9 msg/s (1.0×)
Exp 2C: AMQP Seq. (Relay B)	200	0.586 (total 117.4)	0.197	0.575	0.736	1,703.9/s (2.84×)
Exp 2C: AMQP Batch-50 Probe	**200**	**0.058 (total 11.8)**	**0.007**	**0.060**	**0.065**	**16,949.2/s (28.25×)**

Under a 200-scan burst ($c = 50$, Exp 2B, Fig. 3a), `ScanOutboxRelayService` drains across 4 quantized batches of $B = 50$ rows in 3,817 ms (3.82 s, matching Eq. (3)). In Exp 2C, replacing sequential HTTP Management requests (333.38 ms total) with persistent AMQP `ConfirmChannel` cuts 200-message publish time to 117.38 ms sequentially (Relay B, 2.84×) and 11.80 ms in a pipelined batch-50 micro-benchmark probe (16,949.2 msg/s, 28.25×).

D. Idempotency, FSM Safety & 5-Level Human Control (RQ3)

Across 500 synthetic HTTP calls covering 150 unique keys (Exp 3, Fig. 3b), Dual-Layer Idempotency returned 500/500 (100.0%) HTTP `2xx` while inserting strictly 150/150 `ScanEvent` and `OutboxEvent` records. Layer-1 cache lookup accelerated sequential retries 2.12× (1.825 ms [1.82 ms rounded] vs. 3.873 ms Fresh; median gain 2.05 ms, CI [1.57, 3.95] ms, $p = 9.61 \times 10^{-13}$), while Layer-2 `P2002` resolved every 6-way parallel race (68.09 ms median). Re-injecting 50 duplicate RabbitMQ envelopes yielded 0/50 Tracking leaks and 0/50 clean KPI leaks (1,750 → 1,750, 60/60 distinct `AggregationJob` commits), while SQL audit timestamps explained a transient +50 initial probe offset (`07:51:07.224Z` vs. `10.085Z`, 2.86 s SQL tail under `ack_requeue_false`). Against 60 illegal out-of-order and terminal AMQP attacks (Exp 4A), `ShipmentStateMachine` protected 60/60 (100.0%; exact Clopper–Pearson 95% CI [94.04%, 100.0%]) authoritative states, while Exp 4B (Fig. 3c) verified 5/5 (100.0%; CI [47.82%, 100.0%]) delayed parent faults self-healed on `.retry.10s` at 10.36 s and 5/5 poison events escalated cleanly to `.retry.1m` (17.03 s) and `.dlq` (71.97 s) under our 5-level human-control scale (Table V).

TABLE V
VERIFICATION TELEMETRY (EXP 3–4B) MAPPED TO 5-LEVEL HUMAN-CONTROL SCALE

Verification Scope	Human-Control Scale Tier	Measured Outcome	Verified Mechanism & Audit Finding
Terminal OOO & View (Exp 4A FSM + Track)	L1: Unchecked Auto (Disallowed)	0 FSM jumps; 20/20 View Skew	`VALID_TRANSITIONS` blocks 60/60 attacks; `TrackingCurrent` F1 exposes view skew
Non-Blocking Offload (Exp 4B Main Queue)	L2: Blocking Retry (Disallowed)	0 main-queue stalls	Consumer ACKs main queue immediately; routes retries via `x-retry-count`
Exp 3 / 3C / 4A: Guards & Idempotent Gate	**L3: Auto Guard & Dedup Gate**	**150/150 Exact; 0/50 Dup Leak**	**L1 (1.825 ms, 2.12×) + L2 `P2002` dedup; `AggregationJob` 150/150 (+2.86 s F2)**
Exp 4B: Transient Orphan Self-Healing	**L4: Bounded Auto TTL Retry (10 s / 1 m)**	**5/5 Healed (10.36 s) CI [47.8, 100.0%]**	**`.retry.10s` resolves race after parent create @ 3.09 s (0 human alerts)**
Exp 4B Poison `.dlq` & Exp 4A Lock Gate	**L5: Human Operator Quarantine & Override**	**5/5 `.dlq` (71.97 s) 20/20 Phased Locks**	**Poison isolates in `.dlq`; locked scans override via `INVENTORY_CHECK` (F3)**

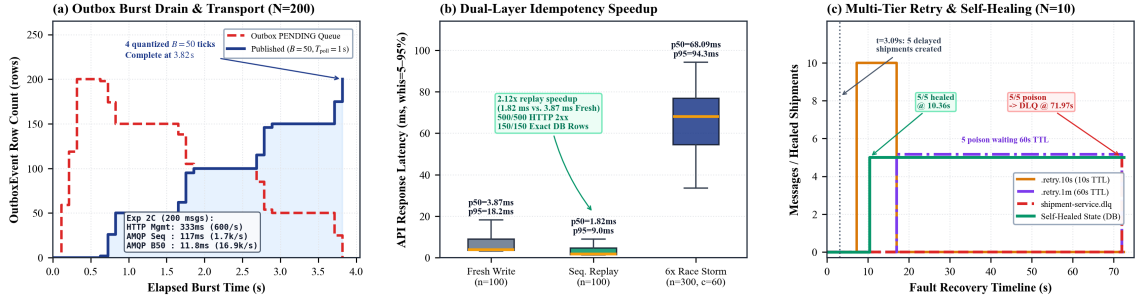


Fig. 3. Burst drain, deduplication, and self-healing telemetry: (a) Outbox Batch-50 burst drain staircase (N = 200 writes, T_{poll} = 1,000 ms) with Exp 2C transport inset; (b) Dual-Layer Idempotency latency speedup (N = 500 requests / 150 unique keys); and (c) Live multi-tier TTL retry recovery and `.dlq` quarantine trajectory over 72.5 s (N = 10 faults).

V. DISCUSSION, CODE FINDINGS & TARGETED REMEDIATIONS

Answering RQ1–RQ3: (1) On RQ1, Outbox + CQRS eliminates shared KPI row-locks above $c = 10$ (783.36 req/s peak) and reduces p_{95} tracking read latency by 65.59% under write saturation (15.73 vs. 45.73 ms, $p = 4.83 \times 10^{-67}$). (2) On RQ2, fixed 1,000 ms polling dwell explains 99.69% of mean post-commit lag (531.96 ms) and 99.98% of variance, whereas pipelined AMQP `ConfirmChannel` probing boosts broker publish throughput up to 28.25× over HTTP Management relays. (3) On RQ3, combining L1/L2 `P2002` idempotency, 19-state FSM whitelisting (60/60), and 5-level human-control `.retry`/`.dlq` ladders enforces safe convergence while preserving human governance.

Three Active Codebase Findings & Targeted Remediations (Table VI): Live fault injection exposed three active engineering gaps in the running codebase, paired below with proposed patches (unapplied during the frozen benchmark sweep):

1) **CQRS Read-Model View Skew (F1):** Inside `TrackingDisplayMapper`, out-of-order events omitting `data.shipment.currentStatus` fall back to event-to-status mapping and overwrite `TrackingCurrent` across all 20/20 test parcels even while `ShipmentService` rejected all 60 attacks. *Fix:* Guard `TrackingCurrent` updates with monotonic `stateVersion` or `occurredAt` checks and derive status strictly from authoritative `shipment.state_changed` events.
2) **HTTP Pull Ackmode Crash Window (F2):** `ReportingRabbitmqConsumerService` pops 20-message batches via `ack_requeue_false` up to 2.86 s ahead of sequential SQL `AggregationJob` commits. *Fix:* Switch to persistent AMQP push with manual `basicAck` executed strictly post-commit.
3) **Inter-Service Lock TOCTOU & Outbox Atomicity (F3):** Interleaved `INVENTORY_CHECK` bursts triggered 18/20 lock probes (vs. 20/20 phased) because `ShipmentsService` clears `isLocked = false` asynchronously mid-tick while `ScansService` queries locks over HTTP and persists `ScanEvent` and `OutboxEvent` sequentially outside `prisma.$transaction`. *Fix:* Replicate lock flags locally into `scan_db` via domain events and wrap writes atomically in `prisma.$transaction`.

TABLE VI
ACTIVE CODEBASE AUDIT FINDINGS AND TARGETED REMEDIATIONS

Finding ID	Active Service Component	Measured Cluster Evidence	Proposed Engineering Remediation
F1: CQRS Read View Skew	**`TrackingDisplayMapper` / `TrackingProjection-Store`**	**20/20 overwritten views on missing-snapshot OOO**	**Guard `TrackingCurrent` by monotonic `occurredAt` / `stateVersion`**
F2: HTTP Pull Crash Window	`ReportingRabbitmq-ConsumerService`	+50 in-flight / 2.86 s lag under `ack_requeue_false`	Switch HTTP `/get` to AMQP manual `basicAck` post-commit
F3: Async Lock TOCTOU	`ScansService` & `ShipmentsService`	18/20 interleaved vs. 20/20 phased; 6.91 ms, $c = 1$ HTTP	Sync lock flag into `scan_db`; wrap writes in `prisma.$transaction`

VI. CONCLUSION, LIMITATIONS, AND REPRODUCIBILITY

Conclusion: Evaluating the live Nexus Transactional Outbox, Dual-Layer Idempotency, 19-State FSM, and CQRS architecture against a shared-database baseline confirms 7.52× write scaling (783.36 req/s), 65.59% lower tracking read p_{95} latency ($p = 4.83 \times 10^{-67}$), 28.25× batched AMQP publish speedup in micro-benchmark probing, 100.0% FSM invariant safety (60/60, [94.04%, 100.0%]), 5/5 transient self-healing at 10.36 s balanced with Level-5 `.dlq` quarantine at 71.97 s, and three concrete active code findings (`F1`–`F3`).

Limitations & Reproducibility: Four boundaries frame our results: (1) Single-host Apple M4 Docker runs ($n = 1$ live sweep, executed `2026-11-09T14:48:54Z` plus follow-up SQL/clean/phased addendum) exclude WAN jitter; (2) Burst suites (100–500 requests) reflect synthetic fault workloads rather than multi-day production traces; (3) At $c = 25$ writes, single-request latency medians do not differ significantly ($p = 0.177$) despite 57.7% lower p_{95} tails, while Exp 4B's 5/5 heal rate spans [47.82%, 100.0%] at $N = 5$; and (4) Findings `F1`–`F3` remain unpatched in evaluated binaries alongside priority upgrades (`LISTEN/NOTIFY` wakeups and pipelined AMQP confirms). Code, frozen JSON datasets (`canonical_stats.json`, raw summaries, and addendum JSONs under `data_distributed/`), and the cluster harness (`run_benchmarks.mjs`, plus Python analysis and figure generators under `docs/research/paper/`) are archived at `https://github.com/VoVanTuTai/logistics-management-system`.

REFERENCES

[1] C. Richardson, *Microservices Patterns: With Examples in Java*. Shelter Island, NY, USA: Manning Publications, 2018 (520 pp., ISBN 978-1-61729-454-9).
[2] D. J. Abadi, "Consistency tradeoffs in modern distributed database system design: CAP is only part of the story," *Computer*, vol. 45, no. 2, pp. 37–42, Feb. 2012.
[3] S. Gilbert and N. Lynch, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, Jun. 2002.
[4] D. Pritchett, "BASE: An Acid alternative," *ACM Queue*, vol. 6, no. 3, pp. 48–55, May 2008.
[5] R. Laigner, Y. Zhou, M. A. V. Salles, Y. Liu, and M. Kalinowski, "Data management in microservices: State of the practice, challenges, and research directions," *Proc. VLDB Endow.*, vol. 14, no. 13, pp. 3348–3361, 2021.
[6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021 (616 pp., ISBN 978-1-4920-3401-8).
[7] V. Vernon, *Implementing Domain-Driven Design*. Upper Saddle River, NJ, USA: Addison-Wesley, 2013 (656 pp., ISBN 978-0-321-83457-7).
[8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston, MA, USA: Addison-Wesley, 2003 (ISBN 978-0-321-20068-6).
[9] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O'Reilly Media, 2017 (616 pp., ISBN 978-1-4493-7332-0).
[10] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A distributed messaging system for log processing," in *Proc. NetDB Workshop*, Athens, Greece, 2011, pp. 1–7.
[11] J. Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction," *LinkedIn Engineering Blog*, Dec. 2013.
[12] A. Videla and J. J. W. Williams, *RabbitMQ in Action: Distributed Messaging for Everyone*. Shelter Island, NY, USA: Manning Publications, 2012 (312 pp., ISBN 978-1-935182-97-9).
[13] G. Young, "CQRS Documents," *CQRS Info Technical Notes*, Nov. 2010.
[14] P. Helland, "Idempotence is not a medical condition," *ACM Queue*, vol. 10, no. 4, pp. 30–46, Apr. 2012.
[15] H. Garcia-Molina and K. Salem, "Sagas," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, San Francisco, CA, USA, 1987, pp. 249–259.
[16] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978.
[17] M. Fowler, "CQRS," *MartinFowler.com Architecture Series*, Jul. 2011.